Name (each course has a color)	Problem	Input Form	Running Time	Paradigm	Notes
Grade School Integer Multiplication	Multiply two integers	2 n-digit numbers x, y	O(log10(x)**2) or O(n**2)	Brute Force	x and y can have different lengths
Karatsuba Muliplication	Multiply two integers	2 n-digit numbers x, y	O(n**log2(3)) or O(n**1.585)	Divide and Conquer	
Merge Sort	Sort array of n numbers	Array of numbers	O(n log2(n))	Divide and Conquer	array could be of comparable items
Inversions Count	Count number of inversions in an array	Array of numbers	O(n log2(n))	Divide and Conquer	all numbers bigger and come before x in the array are inversions of x
Strassen's Subcubic Matrix Multiplication	Multiply two matrices	2 n by n matrices	O(n**log2(7)) or O(n**2.81)	Divide and Conquer	
Closest Pair	Get closest pair distance from list of points in a 2D plain	list of points in a 2D plain	O(n log2(n))	Divide and Conquer	
Quick Sort	Sort array of n numbers	Array of numbers	O(n log2(n))	Divide and Conquer, **Randomization**	using Partitioning around a pivot subroutine
Selection	Return the i-th element of a list	Array of n items, item index i	O(n)	Divide and Conquer, **Randomization**	Could be randomized or deterministic
Queue data structure	Represent list of items	list of items			uses first in first out (FIFO) approach
Stack data structure	Represent list of items	list of items			uses last in first out (IIFO) approach
Graph data structure	Represent graphs	list of notes or vertices and list of edges			Could use Adjacency Matrix or Adjacency Lists
Breadth-First Search (BFS)	Search for item x in Graph G	Graph G(n nodes, m edges), item x	O(n+m)	Brute Force	uses queue to store nodes not yet visited
Shortest Paths with BFS	count shortest path from s to every other node	Graph G(n nodes, m edges), source node s	O(n+m)	Brute Force	works on undirected graphs and all edges have same weight
Undirected Connectivity With BFS	Return the connected parts of the graph	Graph G(n nodes, m edges)	O(n+m)	Brute Force	works on undirected graphs
Depth-First Search (DFS)	Search for item x in Graph G	Graph G(n nodes, m edges), item x	O(n+m)	Brute Force	uses stack to store nodes not yet visited
Topological Sort with DFS	Sort nodes of a graph topologically	Graph G(n nodes, m edges)	O(n+m)	Brute Force	works on directed acylic graphs only
Kosaraju's two-pass algorithm	compute SCCs of the graph G	Graph G(n nodes, m edges)	O(n+m)	Brute Force	works on directed graphs only
Heaps data structure	complete binary tree that satisfies the heap property	array of items with some genius navigation	most operations are O(log2(n))	Divide and Conquer	could be minimum or maximum heaps
Dijkstra's Shortest-path algorithm	count shortest path from s to every other node	Graph G(n nodes, m edges), source node s	O(m log2(n)), wothout heaps O(mn)	Brute Force	works on undirected and directed graphs with non-negative edges
Balanced binary search trees data structure	binary tree that keeps items ordered	node with left and right children	most operations are O(log2(n))	Divide and Conquer	
Hash table data structure	combination of lists and hashing to achieve O(1)	immutable items	on average O(1) or O(log2(n))		Bloom filters is hash table with compression of the input for low RAM like rounters
Scheduling jobs	sort list of jobs to work on them	list of n jobs with weight w, length l	O(n)	Greedy	
PRIM's minimum spanning tree	return the minimum spanning tree of a graph G	Graph G(n nodes, m edges)	O(m log2(n)) using min heaps	Greedy	works on undirected graphs
Kruskal's minimum spanning tree	return the minimum spanning tree of a graph G	Graph G(n nodes, m edges)	O(m log2(n)) using union-find	Greedy	works on undirected graphs
Union-find data structure	representation of partitions of items can quickly unit and find	partitions of n items	O(α(n)) α inverse Ackermann function		With Path Compression & Union by Rank
Huffman Codes	get best binary encoding for a given set of character frequencies	n items with c character, f frequency	O(n log2(n)) with min heaps	Greedy	Prefix-free codes
Maximum Weighted Indpendent Set	get subset of nonadjacent verices of maximum total weight	path graph G(n nodes, w weight)	O(n)	Dynamic Programming	work on path graphs with non-negative weights on verices
Knapsack problem	get subset of items with maximum vallue and weight <= W	capacity W and n items with value v, size s	O(nW)	Dynamic Programming	I need more work on my dynamic programming skills
Sequence Alignment	get the alignment cost of two strings	gap cost g, mismatch cost m, strings x, y	O(len(x) len(y))	Dynamic Programming	that is a good example of the dynamic programming paradigm
Optimal Binary Search tree	compute a valid search tree that minimizes the search time	n items each has frecuency f	O(n**3)	Dynamic Programming	I really need more work on my dynamic programming skills
Bellman-Ford shortest-path algorithm	count shortest path from s to every other node	Graph G(n nodes, m edges), source node s	O(mn) m = n or n**2	Dynamic Programming	works on undirected and directed graphs with no negative cycles
Floyd-Warshall algorithm	all-pairs shortest path, from every node to every other node	Graph G(n nodes, medges)	O(n**2 m), m = n or n**2	Dynamic Programming	works on undirected and directed graphs with no negative cycles
Johnson's all-pairs shortest path algorithm	all-pairs shortest path, from every node to every other node	Graph G(n nodes, medges)	O(mn log2(n)), m = n or n**2	Dynamic Programming	works on undirected and directed graphs with no negative cycles